

Software Development (202001064)
Programming (202001066)

Programming Report – Exploding Kittens

Joris Jacobs 1, s-3290387, j.j.k.jacobs@student.utwente.nl
Joost Liebeton 2, s-3259633, j.c.liebeton@student.utwente.nl

February/2023

Table of Contents

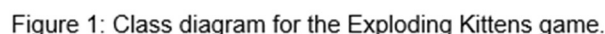
Contents

Overall Design	3
1.1 Class Diagrams	3
1.2 Requirements Implementation Map.....	4
1.3 Model-View-Controller Implementation	5
1.4 User Interface	7
Testing	8
2.1 Unit Testing	8
2.2 System Testing	10
Academic Skills Chapter (Joris Jacobs)	11
3.1 Time Management and Procrastination Avoidance	12
3.2 Strengths and Weaknesses	12
3.3 Checkpoint Meetings	13
Academic Skills Chapter (Joost Liebeton)	13
4.1 Time Management and Procrastination Avoidance	13
4.2 Strengths and Weaknesses	14
4.3 Checkpoint Meetings	14

This project report provides a comprehensive overview of our adaptation of "Exploding Kittens," a popular card game known for its strategic yet whimsical nature. The game involves players drawing cards until someone draws an Exploding Kitten, which eliminates them unless they have a defuse card. Our focus in this report is threefold: Firstly, we will delve into the class diagram of "Exploding Kittens," outlining the key components and their interactions. Secondly, we will discuss the specific requirements that guided the project's development. Finally, we will detail our implementation of the Model-View-Controller (MVC) architecture, highlighting how it enhances the game's functionality and user experience. This introduction sets the stage for a detailed exploration of the game's digital adaptation.

We provide a thorough explanation of the architecture for the card game application "Exploding Kittens" in this class diagram. It carefully plots out the many classes that make up the core of the game, outlining their respective roles and relationships with one another. Classes like PlayerClient and TextIO enable client-side interactions and input/output functions, while EKCHandler, Game, and GameServer establish the foundation for server communication and game logic. Robust error management is ensured by specialized classes such as ExitProgramException, ProtocolException, and ServerUnavailableException, which handle exceptions.

For a clear representation of the game's operation, we have included every class that is a part of the ecosystem—Card, CardType, Deck, Hand, HumanPlayer, Player1—together with the methods and characteristics that go along with them. The text-based user interfaces for the client and server are represented by the ClientTUI and ServerTUI classes in the diagram. Finally, the predefined communication protocols required for client-server interaction are encapsulated in ProtocolMessages. This schematic acts as the fundamental design that encapsulates the spirit and progression of the "Exploding Kittens" game.



1.2 Requirements Implementation Map

In the development of our Exploding Kittens game application, we addressed a comprehensive set of requirements essential for creating and engaging multiplayer experience. The core objectives were centred around enabling gameplay over a network through a client-server model, integrating a user interface for interaction, and ensuring the game adhered to the official rules of Exploding Kittens.

We successfully implemented the fundamental features, allowing players to connect to a server, engage in games, and have a clear announcement of the winner at the conclusion of each match. Our server can host games, maintaining game state, and determining the winner according to the established rules. On the client side, a Text User Interface (TUI) was developed, providing a simple yet functional way for players to interact with the game.

Beyond the basic requirements, we ventured to enhance the player experience by integrating an additional feature – an in-game chat system. This feature fosters player interaction, adding a social dimension to the online gaming experience.

List of implementations with corresponding classes and methods:

1. Network Gameplay

- Classes/Methods:
 - EKHandler class: EKHandler(), handleCommand(), shutdown(),
 - GameServer class: run(), setup(), setupGame(), playCardcmd(),
 - PlayerClient class: start(), createConnection(), clearConnection(), closeConnection(), doConnect()

2. Client Connection and Winner Announcement

- Classes/Methods:
 - Game class: getWinner(),
 - GameServer class: removeClient(), : run(), setup(), setupGame(), playCardcmd(),

3. Server Game Hosting

- Classes/Methods:
 - GameServer class: run(), setup(), setupGame(), playCardcmd(),

4. User Interface (UI/TUI)

- Classes/Methods:
 - ClientTUI class: getString(), start(), printHelpMenu(), handleUserInput(), showMessage()
 - ServerTUI class: ServerTUI(), showMessage(), getBoolean()

5. MVC Architecture

- MVC Breakdown:
 - Model package: Card, CardType, Deck, Hand, HumanPlayer, Player1
 - View package: serverTUI, clientTUI
 - Controller package: EKCHandler, TextIO, Game, GameServer, PlayerClient, ProtocolMessages, Exceptions package.

6. Exception and Error Handling

- Classes/Methods:

- ExitProgramException class: ExitProgramException()
- ProtocolException class: ProtocolException()
- ServerUnavailableException class: ServerUnavailableException()

7. Computer Player

- Classes/Methods:
 - PlayerClient class: readFromServer()

8. Additional Feature: Chat

- Classes/Methods:
 - ClientTUI clas: showMessage(), handleUserInput()
 - EKChandler class: sendMessage()
 - GameServer class: sendChat()
 - PlayerClient class: sendMessage()

1.3 Model-View-Controller Implementation

We used the Model-View-Controller (MVC) design pattern, which is described in our application structure, in the creation of our game. This pattern improves maintainability and scalability in different pieces of the game's functioning.

Model Components: The classes Card, CardType, Deck, and Hand are among the data structures and fundamental game logic that make up the Model in our game. Regardless of user interface concerns, these components reflect the current state of the game and its business logic.

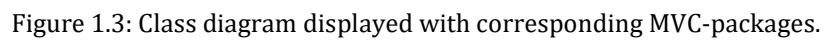
View Components: ClientTUI and ServerTUI are used to implement the view. These parts are in charge of understanding user input and displaying information to the user. We made sure that our user interfaces could be changed or replaced without affecting the core game logic by following the MVC approach.

Controller Components: The Controller communicates between the Model and the View. It consists of classes like Game, GameServer, and PlayerClient. It analyzes user inputs transmitted by the View, modifies the Model as needed, and then refreshes the View.

Figure 1.3's class diagram shows how these MVC components are arranged. As a result of this division, our codebase is easier to read and manage because each component has a clear, defined job.

Linking Controller and Model: In our MVC solution, clearly defined interfaces let the Controller and Model to communicate with one another. This method increases the flexibility and resilience of our application by encouraging loose coupling and high cohesion in addition to streamlining communication between these two layers.

Implementation of the Protocol: The Controller package contains the implementation of the Protocol, which is essential for outlining the guidelines and communication in our game. This arrangement makes sure that the protocol, which is mediated by the Controller and controls how the View and the Model communicate, is in line with the MVC framework.



1.4 User Interface

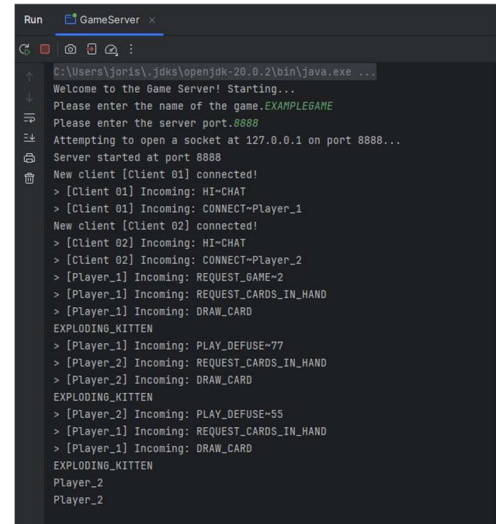
The game's progression is shown in Figures 1.4.1, 1.4.2, and 1.4.3, which show the User Interface (UI) that we implemented for our project. In particular, the Terminal User Interface (TUI) of the game host, also known as the GameServer, is shown in Figure 1.4.1. The TUIs for Player 1 and Player 2 are shown in Figures 1.4.2 and 1.4.3, respectively.

All incoming messages are logged by the GameServer's user interface (UI) in accordance with the protocols that were set at our first design meeting. This function is essential for keeping an eye on how the game is progressing and guaranteeing the dependability of message transfers between the server and clients.

Our TUIs provide a textual assistance menu in an effort to improve user engagement. This menu explains the variety of commands and activities that can be performed in the gaming world, acting as an on-demand guide for players. The idea is to create an interface that is easy to use and comprehend so that players may take advantage of the game's features without needing to read a lot of instructions beforehand.

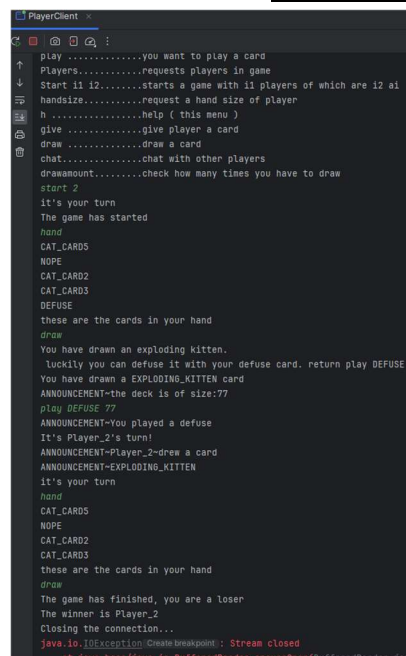
Additionally, the interface is made to provide helpful error messages in response to invalid command inputs. In addition to reducing the possibility of user annoyance, this feedback system helps with the game's instructional component by gently guiding players toward the proper use of instructions.

In conclusion, our project's user interface (UI) is designed to offer a clear, effective, and user-centered gaming experience. This is demonstrated by the organized logging on the GameServer (Figure 1.4.1) and the user-friendly player interfaces (Figures 1.4.2 and 1.4.3) that include instructional elements to support the user's learning.



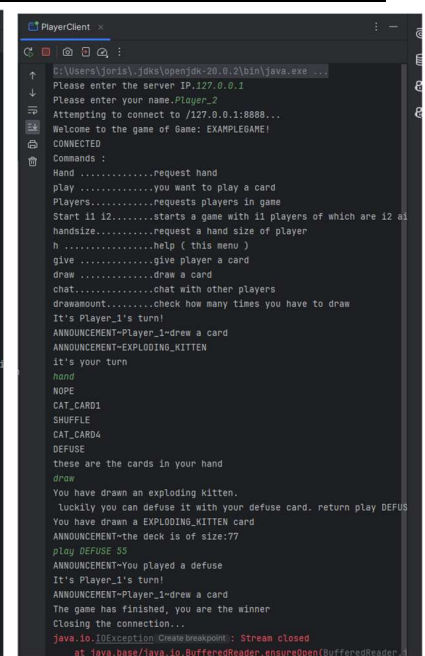
```
Run GameServer
C:\Users\joris\.jdk\openjdk-20.0.2\bin\java.exe ...
Welcome to the Game Server! Starting...
Please enter the name of the game:EXAMPLEGAME
Please enter the server port:8888
Attempting to open a socket at 127.0.0.1 on port 8888...
Server started at port 8888
New client [Client 01] connected!
> [Client 01] Incoming: HI-CHAT
> [Client 01] Incoming: CONNECT-Player_1
New client [Client 02] connected!
> [Client 02] Incoming: HI-CHAT
> [Client 02] Incoming: CONNECT-Player_2
> [Player_1] Incoming: REQUEST_GAME-2
> [Player_1] Incoming: REQUEST_CARDS_IN_HAND
> [Player_1] Incoming: DRAW_CARD
EXPLODING_KITTEN
> [Player_1] Incoming: PLAY_DEFUSE-77
> [Player_2] Incoming: REQUEST_CARDS_IN_HAND
> [Player_2] Incoming: DRAW_CARD
EXPLODING_KITTEN
> [Player_2] Incoming: PLAY_DEFUSE-55
> [Player_1] Incoming: REQUEST_CARDS_IN_HAND
> [Player_1] Incoming: DRAW_CARD
EXPLODING_KITTEN
Player_2
Player_2
```

Figure 1.4.1 Host running a game in GameServer



```
PlayerClient
play .....you want to play a card
Players.....requests players in game
Start 11 12.....starts a game with 11 players of which are 12 at
handsize.....request a hand size of player
h .....help ( this menu )
give .....give player a card
draw .....draw a card
chat.....chat with other players
drawamount.....check how many times you have to draw
start 2
It's your turn
The game has started
hand
CAT_CARDS
NOPE
CAT_CARD2
CAT_CARD3
DEFUSE
these are the cards in your hand
draw
You have drawn an exploding kitten.
luckily you can defuse it with your defuse card. return play DEFUSE 77
You have drawn a EXPLODING_KITTEN card
ANNOUNCEMENT-the deck is of size:77
play DEFUSE 77
ANNOUNCEMENT-You played a defuse
It's Player_2's turn!
ANNOUNCEMENT-Player_2-drew a card
ANNOUNCEMENT-EXPLODING_KITTEN
It's your turn
hand
CAT_CARDS
NOPE
CAT_CARD2
CAT_CARD3
these are the cards in your hand
draw
The game has finished, you are a loser
The winner is Player_2
Closing the connection...
java.io.IOException: Stream closed
at java.base/java.io.BufferedReader.ensureOpen(BufferedReader.java:100)
```

Figure 1.4.2 Player_1 running a game in PlayerClient



```
PlayerClient
play .....you want to play a card
Players.....requests players in game
Start 11 12.....starts a game with 11 players of which are 12 at
handsize.....request a hand size of player
h .....help ( this menu )
give .....give player a card
draw .....draw a card
chat.....chat with other players
drawamount.....check how many times you have to draw
It's Player_1's turn!
ANNOUNCEMENT-Player_1-drew a card
ANNOUNCEMENT-EXPLODING_KITTEN
It's your turn
hand
CAT_CARD1
SHUFFLE
CAT_CARD4
DEFUSE
these are the cards in your hand
draw
You have drawn an exploding kitten.
luckily you can defuse it with your defuse card. return play DEFUS
You have drawn a EXPLODING_KITTEN card
ANNOUNCEMENT-the deck is of size:77
play DEFUSE 33
ANNOUNCEMENT-You played a defuse
It's Player_1's turn!
ANNOUNCEMENT-Player_1-drew a card
The game has finished, you are the winner
Closing the connection...
java.io.IOException: Stream closed
at java.base/java.io.BufferedReader.ensureOpen(BufferedReader.java:100)
```

Figure 1.4.3 Player_2 running a game in PlayerClient

Testing

To plan and implement the testing of our project, we first established clear objectives for what the test should accomplish. This involved identifying key functionalities and features that needed to be verified for proper operation. We then designed a series of tests, both automated and manual, to cover these aspects. These tests were structured to simulate real-world usage scenarios, ensuring that our project would perform reliably under typical user conditions. Regular testing sessions were scheduled throughout the development process, allowing us to identify and rectify any issues that arose. This iterative process made sure that our project was fully tested and had a solid functioning foundation when it was finished.

2.1 Unit Testing

We will briefly go over some of the JUnit tests we wrote.

JUnit PlayerClientTest

Method: handleHello

1. **Strategy Used:** Tested in isolation.
2. **Test Method Applied:** `testHandleHello`.
3. **Test Results and Expected Results:** The test ensures that the `handleHello` method correctly processes a valid handshake response from the server. It expects the method to handle the response without throwing any exceptions.
4. **Test Coverage Determination:** The test covers the behavior of the `handleHello` method when receiving a specific type of server response.

Method: handleHelloProtocolException

1. **Strategy Used:** Tested in isolation.
2. **Test Method Applied:** `testHandleHelloProtocolException`.
3. **Test Results and Expected Results:** This test checks if the `handleHello` method appropriately handles unexpected server responses by throwing a `ProtocolException`.
4. **Test Coverage Determination:** This test case ensures that the method reacts correctly when the server response violates the expected protocol.

Method: handleHelloServerUnavailableException

1. **Strategy Used:** Tested in isolation.
2. **Test Method Applied:** `testHandleHelloServerUnavailableException`.
3. **Test Results and Expected Results:** The test verifies that the `handleHello` method throws a `ServerUnavailableException` when the server is unavailable.
4. **Test Coverage Determination:** This test case confirms the method's behavior when the server connection cannot be established.

Method: requestPlayers

1. **Strategy Used:** Tested in isolation.
2. **Test Method Applied:** `testRequestPlayers`.
3. **Test Results and Expected Results:** This test ensures that the `requestPlayers` method correctly processes the list of players received from the server.
4. **Test Coverage Determination:** The test covers the functionality of the method in retrieving and handling player data from the server.

Method: doConnect

1. **Strategy Used:** Tested in isolation.
2. **Test Method Applied:** **testDoConnect.**
3. **Test Programs Developed:** No additional test programs developed.
4. **Test Results and Expected Results:** This test validates that the **doConnect** method sends the correct connection message to the server and handles the server response accordingly.
5. **Test Coverage Determination:** The test confirms the behavior of the method in establishing a connection with the server.

Method: createConnection-Empty Name

1. **Strategy Used:** Tested in isolation.
2. **Test Method Applied:** **testCreateConnection_EmptyName.**
3. **Test Results and Expected Results:** The test checks if the method correctly handles an empty name input during the connection setup by throwing an **IllegalArgumentException**.
4. **Test Coverage Determination:** This test case examines the method's behavior when encountering invalid user input.

Method: createConnection-Null Name

1. **Strategy Used:** Tested in isolation.
2. **Test Method Applied:** **testCreateConnection_NullName.**
3. **Test Results and Expected Results:** This test verifies that the method properly handles a null name input during the connection setup by throwing an **IllegalArgumentException**.
4. **Test Coverage Determination:** The test evaluates the method's response to null input.

Method: createConnection-IOException

1. **Strategy Used:** Tested in isolation.
2. **Test Method Applied:** **testCreateConnection_IOException.**
3. **Test Results and Expected Results:** The test ensures that the method appropriately handles an **IOException** during the connection process by throwing an **IllegalArgumentException**.
4. **Test Coverage Determination:** This test case examines the method's behavior when encountering an **IOException** during connection establishment.

Junit GameTest

Method: testAddPlayer

1. **Strategy Used:** Tested in isolation.
2. **Test Method Applied:** **testAddPlayer.** This test ensures that players are correctly added to the game, verifying both the increment in the number of players and the correct assignment of player names.
3. **Test Results and Expected Results:** The test verifies that after adding two players, the game's player list size is exactly 2, with player names matching the expected "Player1" and "Player2". This confirms the game's ability to track players correctly.
4. **Test Coverage Determination:** The test covers scenarios related to player addition, ensuring that the game can handle new players being added without issues.

Method: testDrawCard

1. **Strategy Used:** Tested in combination with the **Player** class.
2. **Test Method Applied:** **testDrawCard.** This test checks the **drawCard** functionality, ensuring that a player's hand is correctly updated with a new card.
3. **Test Results and Expected Results:** The test ensures that after drawing a card, the player's hand is not empty and contains exactly one card. This confirms the game's card drawing mechanism functions as intended.

4. **Test Coverage Determination:** This test assesses the **drawCard** method's ability to interact with player objects correctly and modify their state as expected, ensuring comprehensive coverage of card drawing logic.

Method: testEliminatePlayer

1. **Strategy Used:** Tested in isolation.
2. **Test Method Applied:** **testEliminatePlayer**. It verifies the elimination process, ensuring that once a player is eliminated, they are correctly removed from the game's list of players.
3. **Test Results and Expected Results:** The test confirms that after eliminating a player, the game's player list size decreases accordingly, and the eliminated player is no longer present in the list. This indicates the game's elimination process works as intended.
4. **Test Coverage Determination:** This test ensures coverage of the player elimination functionality, assessing the game's ability to handle changes in the number of active players correctly.

JUnit GameServerTest

Method: testSetupGame

1. **Strategy Used:** This method tests the **setupGame** method of the **GameServer** class in isolation.
2. **Test Method Applied:** Unit testing with JUnit is employed for this method.
3. **Test Programs Developed:** The test program creates an instance of **GameServer**, calls the **setupGame** method, and asserts the properties set by the method.
4. **Test Results and Expected Results:** The test verifies that after calling **setupGame**, the **gameName** property is not null.
5. **Test Coverage Determination:** Test coverage is determined by checking if the **setupGame** method initializes necessary properties correctly.

Method: testAddPlayer

1. **Strategy Used:** This method tests the **addPlayer** method of the **GameServer** class in isolation.
2. **Test Method Applied:** Unit testing with JUnit is used.
3. **Test Programs Developed:** The test program sets up a **GameServer**, adds players, and verifies the number of players added.
4. **Test Results and Expected Results:** The test ensures that the correct number of players is added to the game.
5. **Test Coverage Determination:** Test coverage is determined by checking if the **addPlayer** method correctly adds players to the game.

Method: testDrawCard

1. **Strategy Used:** This method tests the **drawCard** method of the **GameServer** class in isolation.
2. **Test Method Applied:** Unit testing with JUnit is employed.
3. **Test Programs Developed:** The test program creates a **GameServer**, adds players, draws a card, and checks if the hand size increases.
4. **Test Results and Expected Results:** The test ensures that the player can draw a card successfully and that the hand size increases.
5. **Test Coverage Determination:** Test coverage is determined by checking if the **drawCard** method functions as expected, including changing the current player.

Method: testPlayCardcmd

1. **Strategy Used:** This method tests the **playCardcmd** method of the **GameServer** class in

isolation.

2. **Test Method Applied:** Unit testing with JUnit is used.
3. **Test Programs Developed:** The test program initializes a **GameServer**, sets up the game, and simulates playing a card by a player.
4. **Test Results and Expected Results:** The test verifies that a player can play a card successfully and that the result contains the expected card type or "nope".
5. **Test Coverage Determination:** Test coverage is determined by checking if the **playCardcmd** method behaves correctly when a player attempts to play a card.

Method: testRequestCardsInHand

1. **Strategy Used:** This method tests the **requestCardsInHand** method of the **GameServer** class in isolation.
2. **Test Method Applied:** Unit testing with JUnit is employed.
3. **Test Programs Developed:** The test program sets up a **GameServer** with a player and cards in its hand, then requests the cards in the player's hand.
4. **Test Results and Expected Results:** The test ensures that the response to requesting cards in hand contains the expected card types.
5. **Test Coverage Determination:** Test coverage is determined by checking if the **requestCardsInHand** method correctly responds with the cards in the player's hand.

2.2 System Testing

From the beginning of the project, we faced a lot of errors. Some were bigger than others. After trying multiple different methods, we can conclude that trial and error and unit testing were most sufficient for us.

Our system testing plan focuses on unforeseen behaviors. After doing a lot of trial and error ourselves, we made sure to let external users play the game. This way we identify unseen errors. When facing a problem, we tried breaking down problems into smaller parts for more manageable testing. This approach ensures comprehensive testing by focusing on both global concepts and individual method functionalities. By doing this we eventually also solved complex problems such as:

Academic Skills Chapter (Joris Jacobs)

3.1 Time Management and Procrastination Avoidance

Time management reflection weeks 2-4:

During weeks 2-4, as documented in the Assignment of Academic Skills, I faced some challenges in managing my time effectively, particularly in completing exercises on time. During this time, I was still getting used to the demands of the course and striking the right balance between all the academic obligations. Despite a structured approach to scheduling and prioritizing tasks, A major obstacle was the amount and difficulty of the exercises. These difficulties provided me with an important teaching moment, which made me rethink and modify my time management techniques to be more productive in the weeks that followed.

Continued time management and adaptation:

As the course progressed, I recognized the need to continuously refine my time management approach, especially when results were not up to my expectations. This involved a more flexible yet disciplined schedule, allowing for adjustments based on task priority and difficulty. To stay focused and productive, I adopted strategies like time-blocking for important work and brief, targeted study sessions. Regular self-assessment and feedback from peers, including my teammate Joost, were crucial in this process. These changes were helpful since I could eventually see gains in my ability to finish tasks more quickly and more expertly.

Time management for the project:

Our final project strategy reflected these time management experiences coming together. From the onset, Joost and I were mindful of the project's scope and the need for early and consistent effort. We established a clear timeline with specific milestones, ensuring that we started on tasks well in advance. Because of the project's complexity, this systematic approach enabled us to set aside enough time for research, brainstorming, and iterative development. Regular check-ins and progress evaluations helped us stay on track and make necessary adjustments. Even though the project was difficult, we were able to meet our goals quickly and effectively because to our teamwork and efficient time management techniques, which laid a solid foundation for the project's successful conclusion.

3.2 Strengths and Weaknesses

Upon analyzation of the Learning Journey the following can be concluded:

Strongest Skills:

1. Object-Oriented Programming: proficiency in inheritance, polymorphism, and the use of ArrayLists, lists, and Maps.
2. JUnit testing: especially proficient in making JUnit tests.
3. Exception Handling: A strong understanding and application in various scenarios.

weaknesses:

1. I/O Streams and Networking: Identified challenges in these areas, signalling a need for more targeted practice.

Academic Skills Assessment:

- Time Management: I improved scheduling and allocating my time over the weeks, ensuring efficient progress in my learning.
- Strengths and Weaknesses Recognition: Ability to accurately identify areas of strength and improvement.
- Peer Feedback: Contribution to group tasks and leveraging of peer strengths.
- Metacognition/Self-Regulated Learning: Use of checkpoint meetings for self-reflection and learning strategy adjustments.

The checkpoint meetings were highly effective in giving insights for improvement and efficiency in my learning. I appreciated the results, as they played a significant role in helping me achieve my goals. In the future I plan to keep a focused progress log for more targeted discussions and set short-term goals between sessions for continuous and measurable improvement.

3.3 Checkpoint Meetings

Throughout the course, I engaged in two checkpoint meetings with my mentor. These meetings were important in steering my learning journey. They served as important ways of receiving feedback and realigning my educational goals, which was beneficial.

These interactions gave better insights of diverse perspectives in my learning journey. My mentor typically provided perspectives that contradicted my assumptions, resulting in a more advanced understanding of the topic. I learned a lot about critical thinking through this approach. For instance, during the initial weeks, I encountered difficulties in comprehending Object-Oriented Programming (OOP). My mentor offered valuable insights and guidance to enhance my understanding and skills in this area.

The self-assessments were an exercise in discipline and focus. They kept me accountable to my learning goals, and the routine of self-reflection helped in developing a more structured approach to my studies. These assessments were not just about tracking progress but also about setting new, more challenging objectives for myself. They helped me staying on track.

Integrating my mentor's feedback into my self-assessment was a lesson in adaptability and open-mindedness. Rather than viewing feedback as criticism, I saw it as an opportunity for growth. This change in perspective was essential to improving my problem-solving abilities since it taught me to approach problems from many perspectives while following the mentor's advice.

Academic Skills Chapter (Joost Liebeton)

4.1 Time Management and Procrastination Avoidance

Time management reflection weeks 2-4:

During the first weeks of this module, specifically weeks 2-4, I was able to effectively manage my time. I was able to prioritize and finish assignments relatively quickly. These weeks were essential for building a strong basis for the demanding work which lay ahead. I was successful in overcoming these early problems with relative ease by sticking to a plan and staying focused on the tasks. This was helpful later on in the course.

Continued time management and adaptation:

As the course progressed, I continued to refine and adapt my time management strategies to overcome the challenges we faced. This phase involved a more dynamic approach to planning and scheduling, ensuring that I could respond flexibly to changing priorities and deadlines. The ability to adapt was important in maintaining a high level of productivity. Regular self-evaluation and a willingness to adjust my methods when necessary, helped me sustain my performance. By reflecting I had full vision on what still needed to be done. This adaptability was not only beneficial in managing my individual workload but also in ensuring effective collaboration on group projects.

Time management for the project:

I quickly discovered that overseeing our final project required more effort than I had anticipated, especially when it came to the networking aspects that involved client-server interactions. Despite the unexpected difficulty, I focused on segmenting the project into manageable tasks and prioritizing them efficiently. We asked for help where needed and tackled the project bit by bit. My dedication to maintain an active and well-organized work culture under pressure played a crucial role in overcoming these obstacles and greatly helped in the project's final outcome.

4.2 Strengths and Weaknesses

After carefully reviewing my Learning Journey, I've gained valuable insights into my competencies and areas needing improvement in this module. My strengths and weaknesses are as follows:

Strengths:

1. Object-Oriented Programming: I showed strong proficiency in key concepts like inheritance, polymorphism, as well as in utilizing ArrayLists, lists, and Maps.
2. Exception Handling: I have a great understanding of its practical applications, demonstrating effective problem-solving skills in various scenarios.

Weaknesses:

1. I/O Streams and Networking: These areas present challenges for me, indicating a need for more focused and intensive practice to enhance my understanding and skill.

In terms of academic skills, my self-analysis revealed:

1. Time Management: Effectiveness in time management throughout the whole module
2. Strengths and Weaknesses Recognition: I can identify my areas of strength and those requiring improvement, which is crucial in my overall development.
3. Peer Feedback: My collaborative skills are strong; we work as a team and ask each other questions when necessary.
4. Metacognition/Self-Regulated Learning: I use checkpoint meetings as opportunities to reflect on and adjust my learning strategies, enhancing my self-regulation in the learning process.

Regarding the usage and improvement of checkpoint meetings:

I used these meetings for receiving feedback and realigning my learning goals, which were very useful. They provided essential guidance to tasks and gave motivation. I could improve on self-reflection and being aware of deadlines this way I can foresee things and include everything in a realistic project schedule. I previously overlooked some tasks and deadlines and learned about them during the checkpoint meetings.

4.3 Checkpoint Meetings

During the course, I had two checkpoint meetings with my mentor, which played an important role in shaping my learning journey. These meetings were instrumental in receiving feedback and realigning my learning goals. They offered guidance on tasks and providing much-needed motivation.

The meetings have had a big impact on my learning. Firstly, they helped in maintaining a clear focus on my objectives. The mentor's feedback was critical in refining my approach, allowing me to address areas that needed improvement. This process encouraged active learning, as I could identify and work on my weaknesses in an effective way.

I attempted to apply the metacognition cycle by doing a self-evaluation. I gained a better understanding of my

personal learning obstacles and advancement through this practice. If I regularly evaluated my knowledge and skills, I would be better able to identify my areas of strength and progress. This recurrent reflection and review approach increased my learning efficacy and deepened my understanding of the material.

Using feedback from my mentor in my self-assessment was a key part of this process. The mentor's insights provided an external perspective, helping me to see aspects of my work that I might have missed. This feedback was particularly valuable in identifying overlooked tasks and deadlines, an area I recognized as needing improvement. Integrating this feedback into my self-assessment allowed me to develop a more Every two weeks, project schedule and improve task management.